

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Steam Sans .'. Atlas — Live Simulation</title>
  <link rel="preconnect" href="https://fonts.googleapis.com" />
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
  <link
href="https://fonts.googleapis.com/css2?family=DM+Mono:wght@300;400;500&family=DM+Sans:wght@200;300;400;500&display=swap" rel="stylesheet" />

  <style>
    /* —— RESET & BASE
    _____ */
    *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

    :root {
      --sanctuary-cream: #F5F1E8;
      --presence-brown: #4A3728;
      --soft-gold:      #d4af37;
      --resonance-gray: #8B7D6B;
      --deep-black:    #0a0a0a;
      --near-white:     #e8e4db;
    }

    html { scroll-behavior: smooth; }

    body {
      background: var(--deep-black);
      color: var(--near-white);
      font-family: 'DM Sans', sans-serif;
      min-height: 100vh;
      overflow-x: hidden;
    }

    /* —— GRAIN OVERLAY
    _____ */
    body::before {
      content: "";
      position: fixed;
      inset: 0;
      background-image: url("data:image/svg+xml,%3Csvg viewBox='0 0 256 256'
xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter id='noise'%3E%3CfeTurbulence

```

```
type='fractalNoise' baseFrequency='0.9' numOctaves='4'  
stitchTiles='stitch'/%3E%3C/filter%3E%3Crect width='100%25' height='100%25'  
filter='url(%23noise)' opacity='0.04'/%3E%3C/svg%3E");  
  pointer-events: none;  
  z-index: 1000;  
  opacity: 0.4;  
}
```

```
/* —— LAYOUT
```

```
*/
```

```
.page-header {  
  padding: 4rem 4rem 2rem;  
  border-bottom: 1px solid rgba(255,255,255,0.06);  
}
```

```
.page-header .eyebrow {  
  font-family: 'DM Mono', monospace;  
  font-size: 0.65rem;  
  letter-spacing: 0.25em;  
  text-transform: uppercase;  
  color: var(--resonance-gray);  
  margin-bottom: 1rem;  
}
```

```
.page-header h1 {  
  font-family: 'DM Sans', sans-serif;  
  font-weight: 200;  
  font-size: clamp(1.8rem, 4vw, 3rem);  
  color: var(--near-white);  
  letter-spacing: -0.01em;  
}
```

```
.page-header .subtitle {  
  font-family: 'DM Mono', monospace;  
  font-size: 0.75rem;  
  color: var(--resonance-gray);  
  margin-top: 0.75rem;  
  letter-spacing: 0.05em;  
}
```

```
.sections-wrapper {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 0;
```

```
}
```

```
.register-section {  
  padding: 5rem 4rem;  
  border-bottom: 1px solid rgba(255,255,255,0.06);  
  position: relative;  
}
```

```
.register-label {  
  font-family: 'DM Mono', monospace;  
  font-size: 0.6rem;  
  letter-spacing: 0.3em;  
  text-transform: uppercase;  
  color: var(--soft-gold);  
  margin-bottom: 0.5rem;  
  opacity: 0.7;  
}
```

```
.register-name {  
  font-family: 'DM Mono', monospace;  
  font-size: 0.75rem;  
  letter-spacing: 0.15em;  
  color: var(--resonance-gray);  
  margin-bottom: 3rem;  
}
```

```
.code-annotation {  
  margin-top: 3rem;  
  padding: 1.5rem 2rem;  
  background: rgba(255,255,255,0.03);  
  border-left: 2px solid rgba(212,175,55,0.3);  
  border-radius: 0 4px 4px 0;  
}
```

```
.code-annotation pre {  
  font-family: 'DM Mono', monospace;  
  font-size: 0.7rem;  
  line-height: 1.8;  
  color: var(--resonance-gray);  
  white-space: pre-wrap;  
  overflow-x: auto;  
}
```

```
.code-annotation .comment { color: rgba(139,125,107,0.5); }
```

```
.code-annotation .property { color: #7ec8e3; }  
.code-annotation .value { color: #d4af37; }  
.code-annotation .selector { color: #c9a96e; }
```

```
/* ——— INTERACTIVE INPUT
```

```
*/
```

```
.input-row {  
  display: flex;  
  align-items: center;  
  gap: 1rem;  
  margin-bottom: 1rem;  
}
```

```
.input-row label {  
  font-family: 'DM Mono', monospace;  
  font-size: 0.65rem;  
  letter-spacing: 0.15em;  
  color: var(--resonance-gray);  
  white-space: nowrap;  
}
```

```
.text-input {  
  background: transparent;  
  border: none;  
  border-bottom: 1px solid rgba(255,255,255,0.1);  
  color: inherit;  
  font-size: inherit;  
  font-family: inherit;  
  outline: none;  
  width: 100%;  
  padding: 0.25rem 0;  
}
```

```
/* ——— REGISTER 1: HARRIS
```

```
*/
```

```
#harris-section { background: #0a0a0a; }
```

```
.harris-display {  
  font-family: 'DM Sans', sans-serif;  
  font-weight: 300;  
  font-size: clamp(2.5rem, 8vw, 7rem);  
  letter-spacing: 0.12em;  
  text-transform: uppercase;  
  color: var(--near-white);
```

```

line-height: 1;
position: relative;
}

/* Grid underlay */
.harris-grid {
position: absolute;
inset: -1rem;
background-image:
  linear-gradient(rgba(255,255,255,0.03) 1px, transparent 1px),
  linear-gradient(90deg, rgba(255,255,255,0.03) 1px, transparent 1px);
background-size: 40px 40px;
pointer-events: none;
}

.harris-meta {
position: absolute;
top: 0.5rem;
right: 0;
font-family: 'DM Mono', monospace;
font-size: 0.55rem;
letter-spacing: 0.1em;
color: rgba(139,125,107,0.4);
text-align: right;
line-height: 1.8;
}

/* —— REGISTER 2: HBA */
#hba-section { background: #0d0c0b; }

.hba-display {
font-family: 'DM Mono', monospace;
font-weight: 300;
font-size: clamp(2rem, 6vw, 5.5rem);
letter-spacing: 0.04em;
color: var(--near-white);
line-height: 1.2;
display: flex;
flex-wrap: wrap;
gap: 0;
}

.hba-char {

```

```
display: inline-block;
position: relative;
animation: hba-breathe var(--duration, 4s) ease-in-out infinite;
animation-delay: var(--delay, 0s);
transform-origin: bottom center;
}
```

```
@keyframes hba-breathe {
  0% { transform: translateY(0) scaleY(1); opacity: 0.90; filter: blur(0px); }
  25% { transform: translateY(-1px) scaleY(1.01); opacity: 1.00; filter: blur(0px); }
  50% { transform: translateY(-2px) scaleY(1.02); opacity: 0.95; filter: blur(0.3px); }
  75% { transform: translateY(-1px) scaleY(1.01); opacity: 1.00; filter: blur(0px); }
  100% { transform: translateY(0) scaleY(1); opacity: 0.90; filter: blur(0px); }
}
```

```
/* Subtle stroke inconsistency via text-shadow */
.hba-char:nth-child(odd) { text-shadow: 0.3px 0 0 rgba(255,255,255,0.05); }
.hba-char:nth-child(even) { text-shadow: -0.2px 0 0 rgba(255,255,255,0.03); }
.hba-char.space { width: 0.35em; }
```

```
/* —— REGISTER 3: VAPOR
```

```
*/
```

```
#vapor-section { background: #080808; }
```

```
.vapor-wrapper {
  position: relative;
  width: 100%;
  min-height: 160px;
}
```

```
#vapor-canvas {
  display: block;
  width: 100%;
  height: 160px;
}
```

```
.vapor-controls {
  display: flex;
  gap: 2rem;
  margin-top: 1.5rem;
  align-items: center;
  flex-wrap: wrap;
}
```

```
.vapor-btn {
  font-family: 'DM Mono', monospace;
  font-size: 0.65rem;
  letter-spacing: 0.2em;
  text-transform: uppercase;
  color: var(--soft-gold);
  background: transparent;
  border: 1px solid rgba(212,175,55,0.3);
  padding: 0.6rem 1.4rem;
  cursor: pointer;
  transition: all 0.4s ease;
}
```

```
.vapor-btn:hover {
  background: rgba(212,175,55,0.08);
  border-color: rgba(212,175,55,0.6);
}
```

```
.vapor-text-input {
  font-family: 'DM Mono', monospace;
  font-size: 0.75rem;
  background: transparent;
  border: none;
  border-bottom: 1px solid rgba(255,255,255,0.1);
  color: var(--near-white);
  padding: 0.4rem 0;
  outline: none;
  width: 280px;
  letter-spacing: 0.1em;
}
```

```
/* —— STABILITY AXIS DEMO
```

```
*/
```

```
#axis-section { background: #0b0b0a; }
```

```
.axis-demo {
  display: grid;
  grid-template-rows: 1fr 1fr 1fr;
  gap: 2.5rem;
}
```

```
.axis-row {
  display: flex;
  align-items: baseline;
```

```
gap: 3rem;
}
```

```
.axis-marker {
  font-family: 'DM Mono', monospace;
  font-size: 0.6rem;
  letter-spacing: 0.15em;
  color: rgba(139,125,107,0.5);
  min-width: 80px;
  line-height: 1;
}
```

```
.axis-marker span {
  display: block;
  color: var(--soft-gold);
  font-size: 0.75rem;
  margin-bottom: 0.2rem;
  opacity: 0.6;
}
```

```
/* Row 0: Harris */
.axis-harris {
  font-family: 'DM Sans', sans-serif;
  font-weight: 300;
  font-size: clamp(1.8rem, 5vw, 4.5rem);
  letter-spacing: 0.14em;
  text-transform: uppercase;
  color: var(--near-white);
}
```

```
/* Row 50: HBA */
.axis-hba {
  font-family: 'DM Mono', monospace;
  font-weight: 300;
  font-size: clamp(1.6rem, 4.5vw, 4rem);
  letter-spacing: 0.06em;
  color: rgba(232,228,219,0.8);
  animation: hba-breathe 6s ease-in-out infinite;
  filter: blur(0.2px);
}
```

```
/* Row 100: Vapor — canvas inline */
.axis-vapor-canvas {
  height: 80px;
```



```
    flex: 1;
  }

/* — SPECIAL GLYPHS */
#glyphs-section { background: #090909; }

.glyphs-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));
  gap: 2rem;
}

.glyph-card {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 1rem;
  padding: 2rem 1rem;
  border: 1px solid rgba(255,255,255,0.05);
  transition: border-color 0.4s ease;
  cursor: default;
}

.glyph-card:hover { border-color: rgba(212,175,55,0.2); }

.glyph-symbol {
  font-size: 3rem;
  line-height: 1;
  color: var(--near-white);
}

.glyph-name {
  font-family: 'DM Mono', monospace;
  font-size: 0.6rem;
  letter-spacing: 0.2em;
  text-transform: uppercase;
  color: var(--resonance-gray);
  text-align: center;
}

.glyph-use {
  font-family: 'DM Mono', monospace;
  font-size: 0.55rem;
}
```

```

    color: rgba(139,125,107,0.5);
    text-align: center;
    line-height: 1.6;
}

/* ——— FINAL PRINCIPLE

```

```

#final-section {
    padding: 8rem 4rem;
    text-align: center;
    background: #060606;
}

.final-phrase {
    font-family: 'DM Sans', sans-serif;
    font-weight: 200;
    font-size: clamp(1rem, 3vw, 2rem);
    letter-spacing: 0.08em;
    line-height: 1.6;
    color: rgba(232,228,219,0.5);
    max-width: 700px;
    margin: 0 auto 3rem;
}

.final-phrase em {
    font-style: italic;
    color: rgba(212,175,55,0.6);
}

/* ——— SCROLLBAR

```

```

::-webkit-scrollbar { width: 4px; }
::-webkit-scrollbar-track { background: var(--deep-black); }
::-webkit-scrollbar-thumb { background: rgba(139,125,107,0.3); border-radius: 2px; }
</style>
</head>
<body>

<!-- ——— PAGE HEADER

```

```

<header class="page-header">
  <p class="eyebrow">Steepervse .'. Typographic System</p>
  <h1>Steam Sans .'. Atlas</h1>
  <p class="subtitle">A Living Typographic Instrument — Simulation Demo</p>

```

</header>

<div class="sections-wrapper">

<!-- — REGISTER 1: HARRIS —————

-->

<section class="register-section" id="harris-section">

<p class="register-label">Register 01</p>

<p class="register-name">Steam Sans .'. Harris — Instrumental</p>

<div style="position: relative; overflow: hidden; padding: 2rem 0;">

<div class="harris-grid"></div>

<div class="harris-meta">

TRACKING +120

WEIGHT 300

BASELINE STABLE

STATE RESOLVED

</div>

<div class="harris-display" id="harris-text">MEASURED</div>

</div>

<div class="input-row" style="margin-top: 2rem;">

<label>EDIT TEXT →</label>

<input class="text-input" id="harris-input" type="text" value="MEASURED"

maxlength="12" />

</div>

<div class="code-annotation">

<pre>/* Steam Sans Harris — Instrumental Register */

.steam-sans-harris {

font-family: 'DM Sans', sans-serif;

font-weight: 300;

letter-spacing: 0.12em; /* expanded tracking */

text-transform: uppercase;

opacity: 1; /* fully resolved — no deformation */

transition: opacity 0.5s ease-in-out;

}</pre>

</div>

</section>

```

<!-- ——— REGISTER 2: HBA —————>
-->
<section class="register-section" id="hba-section">
  <p class="register-label">Register 02</p>
  <p class="register-name">Steam Sans .'. HBA — Witness</p>

  <div class="hba-display" id="hba-text" aria-label="WITNESSED"></div>

  <div class="input-row" style="margin-top: 2rem;">
    <label>EDIT TEXT →</label>
    <input class="text-input" id="hba-input" type="text" value="WITNESSED" maxlength="14"
  />
  </div>

  <div class="code-annotation">
    <pre><span class="comment">/* Steam Sans HBA — Witness Register
Each character is wrapped in a span and animated
independently with staggered delays */</span>

<span class="selector">.hba-char</span> {
  <span class="property">display</span>: <span class="value">inline-block</span>;
  <span class="property">animation</span>: <span class="value">hba-breathe var(--duration)
ease-in-out infinite</span>;
  <span class="property">animation-delay</span>: <span class="value">var(--delay)</span>;
}

<span class="selector">@keyframes hba-breathe</span> {
  0% { <span class="property">transform</span>: <span class="value">translateY(0)
scaleY(1)</span>; <span class="property">opacity</span>: <span class="value">0.90</span>;
}
  50% { <span class="property">transform</span>: <span class="value">translateY(-2px)
scaleY(1.02)</span>; <span class="property">opacity</span>: <span
class="value">0.95</span>; }
  100% { <span class="property">transform</span>: <span class="value">translateY(0)
scaleY(1)</span>; <span class="property">opacity</span>: <span class="value">0.90</span>;
}
}

<span class="comment">/* JS: stagger each character's animation */</span>
chars.forEach((char, i) => {
  char.style.setProperty(<span class="value">'--delay'</span>, `${i * 0.12}s`);
  char.style.setProperty(<span class="value">'--duration'</span>, `${3.5 + Math.random() * 2}s`);
});</pre>
</div>

```

</section>

<!-- — REGISTER 3: VAPOR —————

-->

<section class="register-section" id="vapor-section">

<p class="register-label">Register 03</p>

<p class="register-name">Steam Sans .'. Vapor — Transmission</p>

<div class="vapor-wrapper">

<canvas id="vapor-canvas"></canvas>

</div>

<div class="vapor-controls">

<input class="vapor-text-input" id="vapor-input" type="text" value="RELEASED" maxlength="14" placeholder="TYPE TO FORM..." />

<button class="vapor-btn" id="vapor-form-btn">FORM .'.</button>

<button class="vapor-btn" id="vapor-dissolve-btn">DISSOLVE ~</button>

</div>

<div class="code-annotation">

<pre>/* Steam Sans Vapor — Transmission Register
Text is rendered to a hidden canvas, pixels are
extracted and converted to particle objects */

// 1. Render text to offscreen canvas to get pixel data

offCtx.font = 'bold 120px DM Sans';

offCtx.fillText(text, 0, 120);

const pixels = offCtx.getImageData(0, 0, w, h).data;

// 2. Sample visible pixels as particle origins

for (let i = 0; i < pixels.length; i += gap * 4) {

if (pixels[i + 3] > 128) {

particles.push(new Particle(x, y));

}

}

// 3. Each particle has a home position and a

// scattered position. Animate between them.

class Particle {

update(state) {

// state: 'form' → move to home, 'dissolve' → scatter

this.x += (this.targetX -

this.x) * 0.08;

```

    <span class="property">this</span>.y += (<span class="property">this</span>.targetY - <span
class="property">this</span>.y) * 0.08;
  }
}</pre>
</div>
</section>

```

```

<!-- ——— STABILITY AXIS —————>
-->

```

```

<section class="register-section" id="axis-section">
  <p class="register-label">Variable Axis</p>
  <p class="register-name">Stability Axis — 0 (Rigid) → 50 (Drift) → 100 (Dissolve)</p>

```

```

<div class="axis-demo">
  <div class="axis-row">
    <div class="axis-marker"><span>0</span>RIGID</div>
    <div class="axis-harris">ATLAS</div>
  </div>
  <div class="axis-row">
    <div class="axis-marker"><span>50</span>DRIFT</div>
    <div class="axis-hba" id="axis-hba-word"></div>
  </div>
  <div class="axis-row">
    <div class="axis-marker"><span>100</span>DISSOLVE</div>
    <canvas class="axis-vapor-canvas" id="axis-vapor-canvas"></canvas>
  </div>
</div>

```

```

<div class="code-annotation">
  <pre><span class="comment">/* Future: when the real variable font exists,
all three states collapse to a single CSS rule */</span>

```

```

<span class="selector">.steam-sans-atlas</span> {
  <span class="property">font-family</span>: <span class="value">'Steam Sans Atlas'</span>,
sans-serif;
  <span class="property">font-variation-settings</span>:
    <span class="value">'STBL'</span> 50, <span class="comment">/* Stability: 0=Harris,
100=Vapor */</span>
    <span class="value">'PRSS'</span> 25, <span class="comment">/* Pressure: stroke density
*/</span>
    <span class="value">'COHR'</span> 100, <span class="comment">/* Coherence: legibility
*/</span>
    <span class="value">'DRFT'</span> 10; <span class="comment">/* Drift: baseline instability
*/</span>

```

```
<span class="property">transition</span>: <span class="value">font-variation-settings 0.8s ease-out</span>;
}</pre>
</div>
</section>
```

```
<!-- ——— SPECIAL GLYPHS —————>
-->
<section class="register-section" id="glyphs-section">
  <p class="register-label">Steepinverse Glyph Set</p>
  <p class="register-name">Special Marks — The Sacred Alphabet</p>

  <div class="glyphs-grid">
    <div class="glyph-card">
      <div class="glyph-symbol" style="color: var(--soft-gold);">.</div>
      <div class="glyph-name">Therefore Mark</div>
      <div class="glyph-use">Primary symbol of transformation. Separator or emphasis.</div>
    </div>
    <div class="glyph-card">
      <div class="glyph-symbol" style="animation: hba-breathe 4s ease-in-out infinite;">~</div>
      <div class="glyph-name">Flow Mark</div>
      <div class="glyph-use">Indicates continuity or breath.</div>
    </div>
    <div class="glyph-card">
      <div class="glyph-symbol" style="letter-spacing: 0.4em; opacity: 0.7;">...</div>
      <div class="glyph-name">Extended Ellipsis</div>
      <div class="glyph-use">Extended hold — longer than standard.</div>
    </div>
    <div class="glyph-card">
      <div class="glyph-symbol" style="font-size: 2.5rem;">⊥</div>
      <div class="glyph-name">Boundary Mark</div>
      <div class="glyph-use">Used for Surface Tension events.</div>
    </div>
    <div class="glyph-card">
      <div class="glyph-symbol" style="opacity: 0.4; font-size: 2.8rem;">○</div>
      <div class="glyph-name">Void Glyph</div>
      <div class="glyph-use">Represents absence / potential. Used sparingly in Vapor mode.</div>
    </div>
  </div>
</section>
```

```
<!-- ——— FINAL PRINCIPLE —————>
-->
```

```
<section id="final-section">
  <p class="final-phrase">
    This typeface must feel like<br>
    it is trying to hold meaning long enough to be read<br>
    before returning to<br>
    <em>signal, breath, and silence.</em>
  </p>
  <div id="final-vapor-canvas-wrapper" style="max-width: 900px; margin: 0 auto;">
    <canvas id="final-canvas" style="width:100%; height: 100px; display: block;"></canvas>
  </div>
</section>
```

```
</div><!-- end sections-wrapper -->
```

```
<!-- ——— JAVASCRIPT
```

```
----- -->
```

```
<script>
//
```

```
// HARRIS: Live text update
//
```

```
const harrisInput = document.getElementById('harris-input');
const harrisText = document.getElementById('harris-text');

harrisInput.addEventListener('input', () => {
  harrisText.textContent = harrisInput.value.toUpperCase() || 'MEASURED';
});

//
```

```
// HBA: Per-character animation with staggered drift
//
```

```
function buildHBA(text, container) {
  container.innerHTML = '';
  [...text].forEach((char, i) => {
    if (char === ' ') {
      const sp = document.createElement('span');
      sp.className = 'hba-char space';
```



```

        container.appendChild(sp);
        return;
    }
    const span = document.createElement('span');
    span.className = 'hba-char';
    span.textContent = char;
    // Each character gets a unique duration and delay for organic feel
    const duration = (3.5 + Math.random() * 2.5).toFixed(2);
    const delay = (i * 0.11 + Math.random() * 0.08).toFixed(2);
    span.style.setProperty('--duration', `${duration}s`);
    span.style.setProperty('--delay', `${delay}s`);
    container.appendChild(span);
  });
}

```

```

const hbaInput = document.getElementById('hba-input');
const hbaText = document.getElementById('hba-text');

```

```

buildHBA('WITNESSED', hbaText);
buildHBA('ATLAS', document.getElementById('axis-hba-word'));

```

```

hbaInput.addEventListener('input', () => {
  buildHBA(hbaInput.value.toUpperCase() || 'WITNESSED', hbaText);
});

```

```

//

```

```

// VAPOR: Particle system
//

```

```

class ParticleSystem {
  constructor(canvas, opts = {}) {
    this.canvas = canvas;
    this.ctx = canvas.getContext('2d');
    this.opts = Object.assign({
      fontSize: 120,
      fontFamily: 'DM Sans',
      fontWeight: '300',
      color: '#e8e4db',
      gap: 3,
      speed: 0.07,
      scatter: 200,

```

```

    }, opts);
    this.particles = [];
    this.state = 'scattered'; // 'forming' | 'formed' | 'dissolving' | 'scattered'
    this.animId = null;
    this._resize();
    this._loop();
  }

```

```

_resize() {
  const rect = this.canvas.getBoundingClientRect();
  this.canvas.width = rect.width * window.devicePixelRatio || 800;
  this.canvas.height = rect.height * window.devicePixelRatio || 160;
  this.ctx.scale(window.devicePixelRatio, window.devicePixelRatio);
  this.W = rect.width || 800;
  this.H = rect.height || 160;
}

```

```

setText(text) {
  this.particles = [];
  const off = document.createElement('canvas');
  const oCtx = off.getContext('2d');
  const fs = this.opts.fontSize;
  off.width = this.W;
  off.height = this.H;

  oCtx.font = `${this.opts.fontWeight} ${fs}px ${this.opts.fontFamily}`;
  oCtx.fillStyle = '#ffffff';
  oCtx.textBaseline = 'middle';

```

```

  const metrics = oCtx.measureText(text);
  const tx = (this.W - metrics.width) / 2;
  const ty = this.H / 2;
  oCtx.fillText(text, tx, ty);

```

```

  const imageData = oCtx.getImageData(0, 0, this.W, this.H);
  const data = imageData.data;
  const gap = this.opts.gap;

```

```

  for (let y = 0; y < this.H; y += gap) {
    for (let x = 0; x < this.W; x += gap) {
      const idx = (y * this.W + x) * 4;
      if (data[idx + 3] > 100) {
        const scatter = this.opts.scatter;
        this.particles.push({

```

```

    homeX: x,
    homeY: y,
    x: x + (Math.random() - 0.5) * scatter * 2,
    y: y + (Math.random() - 0.5) * scatter * 2,
    targetX: x + (Math.random() - 0.5) * scatter * 2,
    targetY: y + (Math.random() - 0.5) * scatter * 2,
    size: Math.random() * 1.2 + 0.4,
    opacity: Math.random() * 0.5 + 0.3,
    speed: this.opts.speed + Math.random() * 0.04,
  });
}
}
}

form() {
  this.state = 'forming';
  this.particles.forEach(p => {
    p.targetX = p.homeX;
    p.targetY = p.homeY;
  });
}

dissolve() {
  this.state = 'dissolving';
  const scatter = this.opts.scatter * 3;
  this.particles.forEach(p => {
    p.targetX = p.homeX + (Math.random() - 0.5) * scatter;
    p.targetY = p.homeY + (Math.random() - 0.5) * scatter;
  });
}

_loop() {
  this.animId = requestAnimationFrame(() => this._loop());
  const ctx = this.ctx;
  ctx.clearRect(0, 0, this.W, this.H);

  this.particles.forEach(p => {
    p.x += (p.targetX - p.x) * p.speed;
    p.y += (p.targetY - p.y) * p.speed;

    // Subtle drift even when formed
    if (this.state === 'forming' || this.state === 'formed') {
      p.x += (Math.random() - 0.5) * 0.3;
    }
  });
}

```

```

    p.y += (Math.random() - 0.5) * 0.3;
  }

  const dist = Math.hypot(p.x - p.targetX, p.y - p.targetY);
  const alpha = this.state === 'dissolving'
    ? Math.min(1, dist / 50) * p.opacity
    : p.opacity;

  ctx.beginPath();
  ctx.arc(p.x, p.y, p.size, 0, Math.PI * 2);
  ctx.fillStyle = `rgba(232, 228, 219, ${alpha})`;
  ctx.fill();
});
}

destroy() {
  cancelAnimationFrame(this.animId);
}
}

// — Main Vapor Canvas —
const vaporCanvas = document.getElementById('vapor-canvas');
const vaporPS = new ParticleSystem(vaporCanvas, { fontSize: 110, gap: 3, scatter: 300 });
vaporPS.setText('RELEASED');
setTimeout(() => vaporPS.form(), 400);

document.getElementById('vapor-form-btn').addEventListener('click', () => vaporPS.form());
document.getElementById('vapor-dissolve-btn').addEventListener('click', () =>
vaporPS.dissolve());

const vaporTextInput = document.getElementById('vapor-input');
let vaporDebounce;
vaporTextInput.addEventListener('input', () => {
  clearTimeout(vaporDebounce);
  vaporDebounce = setTimeout(() => {
    vaporPS.setText(vaporTextInput.value.toUpperCase() || 'RELEASED');
    vaporPS.form();
  }, 300);
});

// — Axis Vapor Canvas —
const axisVaporCanvas = document.getElementById('axis-vapor-canvas');
const axisVaporPS = new ParticleSystem(axisVaporCanvas, {
  fontSize: 72, gap: 3, scatter: 250, speed: 0.05

```

```

});
axisVaporPS.setText('ATLAS');
setTimeout(() => axisVaporPS.form(), 800);
// Continuously re-dissolve for the axis demo
setInterval(() => {
  axisVaporPS.dissolve();
  setTimeout(() => axisVaporPS.form(), 2500);
}, 5000);

// — Final Canvas —
const finalCanvas = document.getElementById('final-canvas');
const finalPS = new ParticleSystem(finalCanvas, {
  fontSize: 52, fontWeight: '200', gap: 2, scatter: 400, speed: 0.04
});
finalPS.setText('SIGNAL .'. BREATH .'. SILENCE');
setTimeout(() => finalPS.form(), 1200);
setInterval(() => {
  finalPS.dissolve();
  setTimeout(() => finalPS.form(), 3500);
}, 7000);

//

```

```

// RESIZE HANDLER
//

```

```

window.addEventListener('resize', () => {
  [vaporPS, axisVaporPS, finalPS].forEach(ps => {
    ps._resize();
  });
  vaporPS.setText(vaporTextInput.value.toUpperCase() || 'RELEASED');
  vaporPS.form();
  axisVaporPS.setText('ATLAS');
  axisVaporPS.form();
  finalPS.setText('SIGNAL .'. BREATH .'. SILENCE');
  finalPS.form();
});
</script>
</body>
</html>

```